



<https://www.linkedin.com/company/quantum-robotics-automation-research-group/?viewAsMember=true>

Artificial Intelligence

CSE-316

(Section: 65-D/H)

Instructor

Dr. M. Akhtaruzzaman

Associate Professor, CSE, DIU

Associate Professor and Head, RME, DIU

akhtaruzzaman.cse@diu.edu.bd;

akhter900@gmail.com

Theory Lecture-13

Perceptron Learning Rules

02. 04, 2026, DIU, Dhaka.

رب زدني علما وارزقني فهما.

My Lord! Advance me in Knowledge and True Understanding.

Perceptron Learning Rule...

General Learning/Training Structure

In learning process,
the weights are updated iteratively by: $w(k) = w(k-1) + \Delta w(k)$

Where the **weight adjustment** $\Delta w(k) = \eta r(k) x(k)$

η is **learning constant**

r is **learning signal**, which is a function of w , x , and desired output, d

For perceptron, r is the difference between the desired and actual output.
Thus,

$$r = d - y$$

Perceptron Learning Rule...

Therefore, the weight increment is $\Delta w_{ij} = \eta[d_j - y_j]x_i$

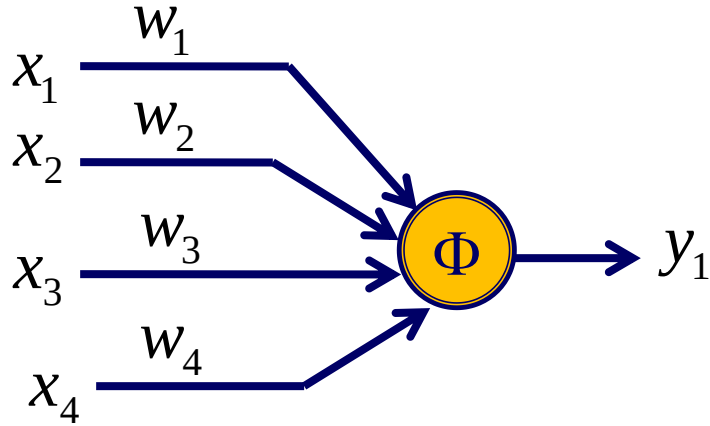
Substituting $y = \Phi(l)$, $\Delta w_{ij} = \eta[d_j - \text{sgn}(\mathbf{w}_i^T \mathbf{x})]x_j$

Since the desired output d_i takes values of ± 1 , we have

$$\Delta w_{ji} = \eta[d_i - \text{sgn}(\mathbf{w}_i^T \mathbf{x})]x_j = \begin{cases} 2\eta d_i x_j, & y_i \neq d_i \\ 0, & \text{otherwise} \end{cases}$$

Perceptron Learning Rule...

Example



Learning constant, $\eta = 0.1$

Initial weights,

$$\mathbf{w} = [1 \quad -1 \quad 0 \quad 0.5]^T$$

Inputs and desired outputs,

$$\mathbf{x}(1) = [1 \quad -2 \quad 0 \quad 1]^T, d(1) = -1$$

$$\mathbf{x}(2) = [0 \quad 1.5 \quad -0.5 \quad 1]^T, d(2) = -1$$

$$\mathbf{x}(3) = [-1 \quad 1 \quad 0.5 \quad 1]^T, d(3) = 1$$

Adjust the weights using the three sets of training data

Perceptron Learning Rule...

Learning process

Step 1:

$$I(1) = \mathbf{w}(1)^T \mathbf{x}(1) = [1 \quad -1 \quad 0 \quad 0.5] \begin{bmatrix} 1 \\ -2 \\ 0 \\ 1 \end{bmatrix} = 3.5$$

$$y(1) = \Phi(I(1)) = \text{sgn}(3.5) = 1$$

$$r(1) = d(1) - y(1) = -1 - 1 = -2$$

$$\begin{aligned} \mathbf{w}(2) &= \mathbf{w}(1) + 0.1(-2)\mathbf{x}(1) \\ &= \mathbf{w}(1) - 0.2\mathbf{x}(1) \end{aligned} = \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{bmatrix} - 0.2 \begin{bmatrix} 1 \\ -2 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.8 \\ -0.6 \\ 0 \\ 0.3 \end{bmatrix}$$

Perceptron Learning Rule...

Learning process

Step 2: $I(2) = \mathbf{w}(2)^T \mathbf{x}(2) = [0.8 \quad -0.6 \quad 0 \quad 0.3] \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ 1 \end{bmatrix} = -0.6$

$$y(2) = \text{sgn}(-0.6) = -1$$

$$r(2) = -1 - (-1) = 0$$

$$\Delta \mathbf{w}(2) = 0, \text{ and thus } \mathbf{w}(3) = \mathbf{w}(2)$$

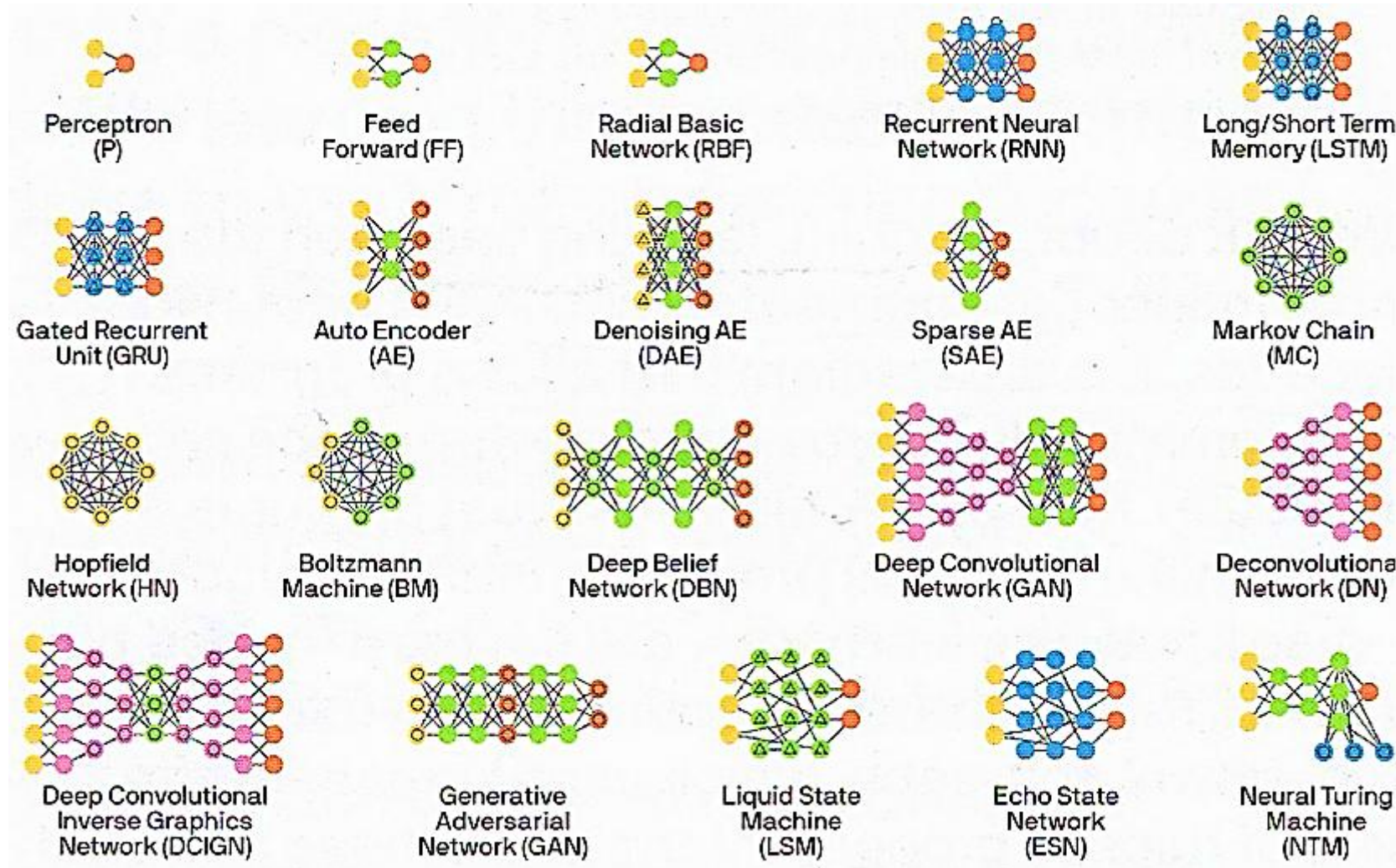
Step 3: $\rightarrow r(3) = 1 - (-1) = 2$

Repeat as above and convince yourselves that finally the weights should be after 3 iterations

$$\mathbf{w}(4) = \begin{bmatrix} 0.6 \\ -0.4 \\ 0.1 \\ 0.5 \end{bmatrix}$$

Step 4: Optional – but try use these weights with the original inputs, does it work right?

Types of ANN



Advantages & Disadvantages of ANN

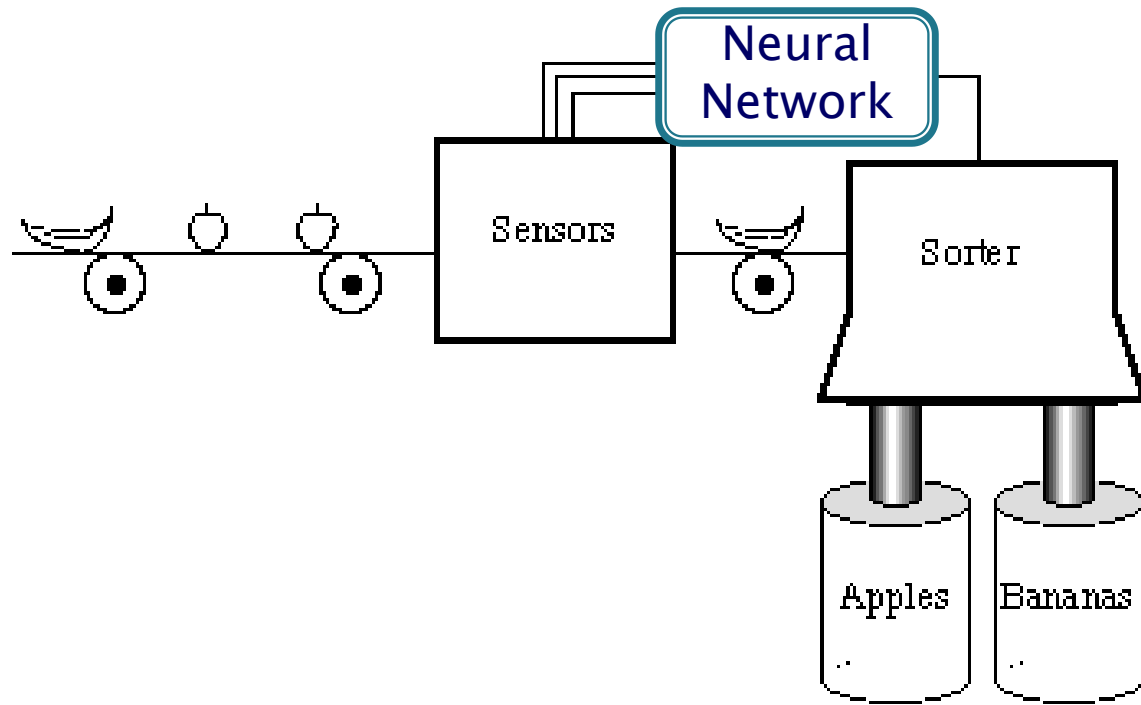
Advantages:

- A neural network can perform tasks in which a linear program cannot.
- When an element of the neural network fails, it can continue without any problem by their parallel nature.
- A neural network does not need to be reprogrammed as it learns itself.
- It can be implemented in an easy way without any problem.
- As adaptive, intelligent systems, neural networks are robust and excel at solving complex problems.
- Neural networks are efficient in their programming and the scientists agree that the advantages of using ANNs outweigh the risks.
- It can be implemented in any application.

Disadvantages:

- The neural network requires training to operate.
- Requires high processing time for large neural networks.
- The architecture of a neural network is different from the architecture and history of microprocessors, so they have to be emulated.

Example – *Fruit Sorting Machine*



Network input

$$x = \begin{bmatrix} shape \\ texture \\ weight \end{bmatrix}$$

Perceptron Learning Rule

$$\text{ANN output} \quad \text{shape} = \begin{cases} 1 & \text{if round} \\ -1 & \text{if elliptical} \end{cases}$$

$$\text{texture} = \begin{cases} 1 & \text{if smooth} \\ -1 & \text{if rough} \end{cases}$$

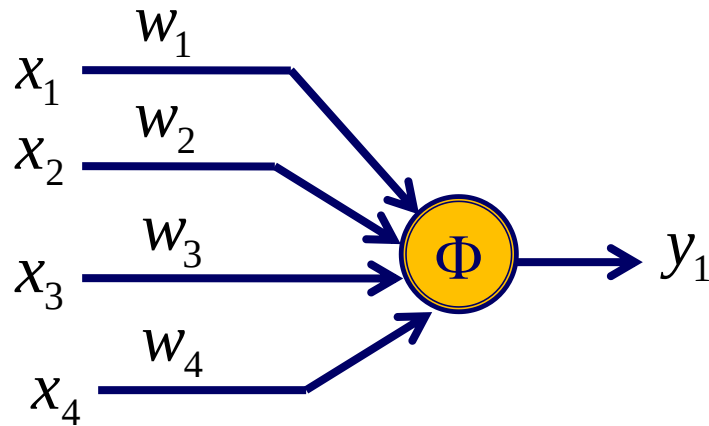
$$\text{weight} = \begin{cases} 1 & \text{if } > 1 \text{ kg} \\ -1 & \text{if } < 1 \text{ kg} \end{cases}$$

Prototype of fruits for learning

$$\text{banana} = \begin{bmatrix} \text{elliptical} \\ \text{smooth} \end{bmatrix} = 1 \quad \text{apple} = \begin{bmatrix} \text{round} \\ \text{rough} \end{bmatrix} = -1$$

Perceptron Learning Rule

Perceptron



Learning constant, $\eta = 0.1$

Initial weights,

$$\mathbf{w} = [0.5 \quad -1 \quad -0.5 \quad 0.5]^T$$

Inputs and its desired outputs

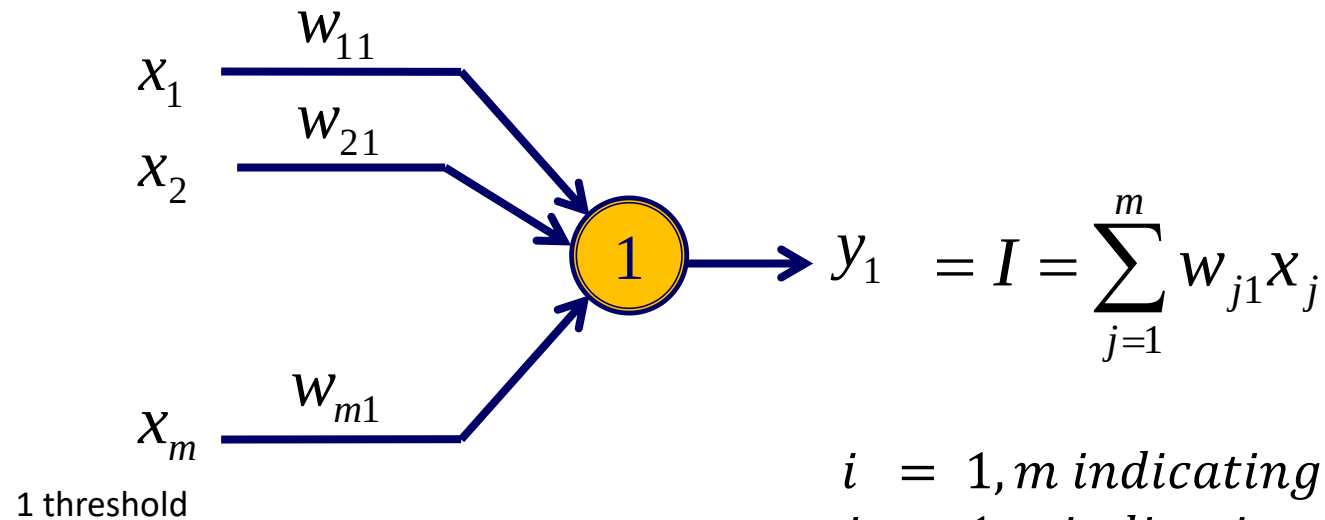
$$\left(\mathbf{x}(1) = \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \end{bmatrix}, d(1) = 1 \right) \begin{array}{l} \text{apple} \\ \text{banana} \end{array} \rightarrow \left(\mathbf{x}(2) = \begin{bmatrix} 1 \\ -1 \\ 1 \\ 1 \end{bmatrix}, d(2) = -1 \right)$$

Adaline Learning Rule

Adaptive Linear Element (adaline)

Neurons that use **single linear activation functions**

For the first neuron / node



$i = 1, m$ indicating inputs

$j = 1, n$ indicating nodes

$k = 1, p$ indicating sequence

ADALINE Learning Rule

General Learning/Training Structure

Update the weights are updated iteratively by:

$$w(k) = w(k-1) + \Delta w(k)$$

$$\text{where } \Delta \mathbf{w}(k) = \eta r(k) \mathbf{x}(k)$$

η is learning constant

r is learning signal, which is a function of w , x , and desired output, d .

The goal of learning is to find the correct weights such that the actual output, y match the desired output, d .

$$y = \sum_{j=1}^m w_{j1} x_j = d$$

ADALINE Learning Rule

Gradient-Descent Algorithm

Define a cost function that relates E to the weights as

$$E(\mathbf{w}) = \frac{1}{2} \sum_{k=1}^p (d(k) - y(k))^2 = \frac{1}{2} \sum (d(k) - \mathbf{w}^T \mathbf{x}(k))^2$$

To minimize error, adjust the weights such that $E(\mathbf{w}) \rightarrow 0$

Adjust each weights by an amount proportional to the negative gradient, hence the name gradient-descent

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} E(\mathbf{w})$$

Where the gradient is calculated as $\nabla_{\mathbf{w}} E(\mathbf{w}) = \left[\frac{\partial E}{\partial w_1} \frac{\partial E}{\partial w_2} \dots \frac{\partial E}{\partial w_m} \right]^T$

ADALINE Learning Rule

Gradient-Descent Algorithm

Therefore the final adjustment for each weight after all data points have been exhausted becomes

$$\Delta w_j = \eta \frac{\partial E}{\partial w_j} = \eta \sum_{k=1}^p (d(k) - \mathbf{w}^T \mathbf{x}(k)) x(k)_j$$

Or individually at each data points

$$\Delta w(k)_j = \eta \frac{\partial E}{\partial w_j} = \eta (d(k) - \mathbf{w}^T(k) \mathbf{x}(k)) x(k)_j$$

Which can be written in vector notation as

$$\Delta \mathbf{w} = \eta \underbrace{(d(k) - \mathbf{w}(k)^T \mathbf{x}(k))}_r \mathbf{x}(k)$$

This learning rule is also known as ADALINE Learning Rule, Widrow-Hoff Learning Rule, and Least Mean Square (LMS) Rule

ADALINE Learning Rule

Filling-in the gap...

Fill in the mathematics to go from

$$E(\mathbf{w}) = \frac{1}{2} \sum_{k=1}^p (d^k - y^k)^2 = \frac{1}{2} \sum_{k=1}^p (d^k - \mathbf{w}^T \mathbf{x}^k)^2$$

through $\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} E(\mathbf{w})$

to $\Delta w_j^k = \eta \frac{\partial E}{\partial w_j} = \eta (d^k - \mathbf{w}^T \mathbf{x}^k) x_j^k$

or $\Delta \mathbf{w} = \eta (d^k - \mathbf{w}^T \mathbf{x}^k) \mathbf{x}^k$

or $\Delta w_j = \eta \frac{\partial E}{\partial w_j} = \eta \sum_{k=1}^p (d^k - \mathbf{w}^T \mathbf{x}^k) x_j^k$

ADALINE Learning Rule

Filling in the gap...

$$E(w) = \frac{1}{2} \sum_{k=1}^p (d^k - y^k)^2 = \frac{1}{2} \sum_{k=1}^p (d^k - w^T x^k)^2$$

$$\Delta w = -\eta \nabla_w E(w) \quad \text{where} \quad \nabla_w E(w) = \left[\frac{\partial E}{\partial w_1} \frac{\partial E}{\partial w_2} \dots \frac{\partial E}{\partial w_m} \right]^T$$

Recall Chain Rule. If $F=f(g(x))$, then $F'=f'(g(x))g'(x)$

$$\text{Thus} \quad \frac{\partial E}{\partial w_1} = \frac{2}{2} \sum_{k=1}^p (d^k - y^k)(-x^k) = -\sum_{k=1}^p (d^k - w^T x^k) x^k$$

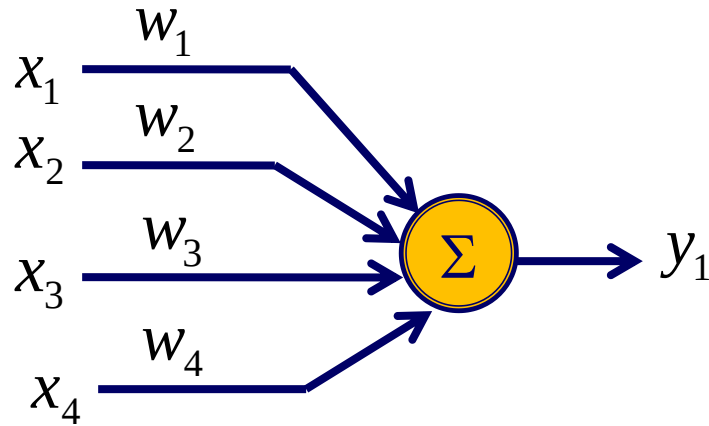
$$\text{thus} \quad \Delta w = \eta (d^k - w^T x^k) x^k$$

$$\text{or} \quad \Delta w_j = \eta \frac{\partial E}{\partial w_j} = \eta \sum_{k=1}^p (d^k - w^T x^k) x_j^k$$

1. Constant Rule : $\frac{d}{dx}(c) = 0$
2. Constant Multiple Rule : $\frac{d}{dx}[cf(x)] = cf'(x)$
3. Power Rule : $\frac{d}{dx}(x^n) = nx^{n-1}$
4. Sum Rule : $\frac{d}{dx}[f(x) + g(x)] = f'(x) + g'(x)$
5. Difference Rule : $\frac{d}{dx}[f(x) - g(x)] = f'(x) - g'(x)$
6. Product Rule : $\frac{d}{dx}[f(x)g(x)] = f(x)g'(x) + g(x)f'(x)$
7. Quotient Rule : $\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{g(x)f'(x) - f(x)g'(x)}{[g(x)]^2}$
8. Chain Rule : $\frac{d}{dx} f[g(x)] = f'[g(x)]g'(x)$

ADALINE Learning Rule

Example



Learning constant, $\eta = 0.1$

Initial weights,

$$\mathbf{w} = [1 \quad -1 \quad 0 \quad 0.5]^T$$

Inputs and desired outputs,

$$\mathbf{x}(1) = [1 \quad -2 \quad 0 \quad 1]^T, \quad d(1) = -1$$

$$\mathbf{x}(2) = [0 \quad 1.5 \quad -0.5 \quad 1]^T, \quad d(2) = -1$$

$$\mathbf{x}(3) = [-1 \quad 1 \quad 0.5 \quad 1]^T, \quad d(3) = 1$$

Adjust the weights using the three sets of training data

ADALINE Learning Rule

Learning process

Step 1:

$$y(1) = \mathbf{w}(1)^T \mathbf{x}(1) = [1 \quad -1 \quad 0 \quad 0.5] \begin{bmatrix} 1 \\ -2 \\ 0 \\ 1 \end{bmatrix} = 3.5 \rightarrow 1, \quad d(1) = -1$$

Thus, needs adjustment

$$\Delta \mathbf{w}(k) = \eta (d(k) - y(k)) \mathbf{x}(k)_j = 0.1(-2) \begin{bmatrix} 1 \\ -2 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.2 \\ 0.4 \\ 0 \\ -0.2 \end{bmatrix}$$

$$\mathbf{w}(2) = \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{bmatrix} + \begin{bmatrix} -0.2 \\ 0.4 \\ 0 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 0.8 \\ -0.6 \\ 0 \\ 0.3 \end{bmatrix}$$

Continue Step 2 and 3 & compare with perceptron

Delta Learning Rule

We have dealt with Perceptron with hard-limiting activation function, and ADALINE with single linear function. Next, look at neuron model with **continuous and differentiable activation function** such as sigmoid.

There will be many neurons, thus recall the indices:

$I = 1, m$ inputs;

$j = 1, n$ nodes; and

$k = 1, p$ training sets

The cost function is,

$$E(w) = \frac{1}{2} \sum_{k=1}^p \sum_{i=1}^m [d(k)_j - \Phi(I(k)_j)]^2$$

Minimize the cost function by gradient-descent method

Delta Learning Rule

Applying the gradient-descent method. Thus, for any particular node and input, and using Chain Rule

$$\frac{\partial E}{\partial w_{ij}} = - \sum_{k=1}^p [d(k)_j - \Phi(I(k)_j)] \Phi'(I(k)_j) x(k)_i$$

$$\text{where } \Phi'(I(k)_j) = \frac{\partial \Phi(I(k)_j)}{\partial I(k)_j}$$

Hence, the weight correction after k-th data set/ pattern is

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}} = \eta \underbrace{[d(k)_j - \Phi(I(k)_j)] \Phi'(I(k)_j)}_r x(k)_i$$

Delta Learning Rule

To use Delta learning rule we need a derivative of the activation function $\Phi(I)$

For unipolar sigmoid function

$$\Phi(I(k)) = \frac{1}{1 + e^{-I(k)}}, \quad \Phi'(I(k)) = y(k)(1 - y(k))$$

And for bipolar sigmoid function

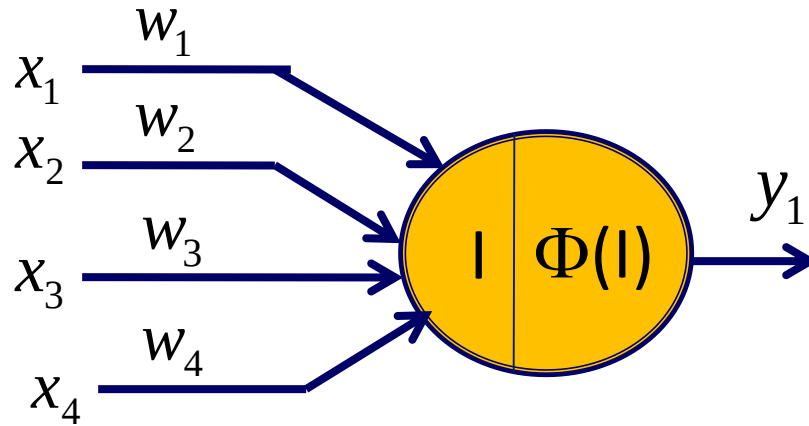
$$\Phi(I(k)) = \frac{2}{1 + e^{-I(k)}} - 1, \quad \Phi'(I(k))' = \frac{1}{2}(1 - y(k)^2)$$

$$\begin{aligned} y &= \frac{1}{1 + e^{-x}} \\ \frac{dy}{dx} &= -\frac{1}{(1 + e^{-x})^2}(-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2} \\ &= \frac{1}{1 + e^{-x}} \left(1 - \frac{1}{1 + e^{-x}} \right) = y(1 - y) \end{aligned}$$

$$\begin{aligned} f(x) &= e^{kx} \\ f'(x) &= k \cdot e^{kx} \end{aligned}$$

Delta Learning Rule

Example



Learning constant, $\eta = 0.1$

Activation function = unipolar sigmoid

Initial weights,

$$\mathbf{w} = [1 \quad -1 \quad 0 \quad 0.5]^T$$

Inputs and desired outputs,

$$\mathbf{x}(1) = [1 \quad -2 \quad 0 \quad 1]^T, \quad d(1) = -1$$

$$\mathbf{x}(2) = [0 \quad 1.5 \quad -0.5 \quad 1]^T, \quad d(2) = -1$$

$$\mathbf{x}(3) = [-1 \quad 1 \quad 0.5 \quad 1]^T, \quad d(3) = 1$$

Adjust the weights using the three sets of training data

Delta Learning Rule

Example

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \Delta \mathbf{w}(k)$$

$$\Delta \mathbf{w}(k) = \eta(d(k) - y(k))\Phi'(I(k))\mathbf{x}(k)$$

$$I(k) = \mathbf{w}^T(k)\mathbf{x}(k)$$

Step 1:

$$I(1) = \begin{bmatrix} 1 & -1 & 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 0 \\ 1 \end{bmatrix} = 3.5$$

$$y(1) = 1/(1 + e^{-3.5}) = 0.971$$

$$d(1) - y(1) = -1.971$$

$$\Phi'(I(1)) = y(1)(1 - y(1)) = 0.971 \times 0.029 = 0.028$$

$$\Delta \mathbf{w}(1) = 0.1 \times (-1.971) \times 0.028 \mathbf{x}(1) = -0.006 \mathbf{x}(1)$$

$$= \begin{bmatrix} -0.006 & 0.012 & 0 & -0.006 \end{bmatrix}^T$$



$$\mathbf{w}(2) = \begin{bmatrix} 1 \\ -1 \\ 0 \\ .5 \end{bmatrix} + \begin{bmatrix} -0.006 \\ 0.012 \\ 0 \\ -0.006 \end{bmatrix} = \begin{bmatrix} 0.994 \\ -0.988 \\ 0 \\ 0.494 \end{bmatrix}$$

Delta Learning Rule

Example...

Step 2:

$$I(2) = [0.994 \quad -0.988 \quad 0 \quad 0.494] \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ 1 \end{bmatrix} = -0.988$$

$$y(2) = 1/(1 + e^{-0.988}) = 0.729$$

$$d(2) - y(2) = -1.729$$

$$\Phi'(I(2)) = y(2)(1 - y(2)) = 0.729 \times 0.271 = 0.198$$

$$\begin{aligned} \Delta \mathbf{w}(2) &= 0.1 \times (-1.729) \times 0.198 \times \mathbf{x}(2) = -3.458 \mathbf{x}(2) \\ &= [0 \quad -5.187 \quad 1.729 \quad -3.458]^T \end{aligned}$$


$$\mathbf{w}(3) = \begin{bmatrix} 0.994 \\ -0.988 \\ 0 \\ 0.494 \end{bmatrix} + \begin{bmatrix} 0 \\ -5.187 \\ 1.729 \\ -3.458 \end{bmatrix} = \begin{bmatrix} 0.994 \\ -6.175 \\ -1.729 \\ -2.964 \end{bmatrix}$$

Delta Learning Rule

Example...

Step 3: do yourselves to get $w(4)$

Summary

Learning Rules

In general, the learning rule is given by $w(k+1) = w(k) + dw(k)$

What differs between the various learning rules is the expression for $dw(k)$, since it involves the calculation of the error between desired and actual output.

Perceptron: $\Delta \mathbf{w} = \eta(d - y)\mathbf{x}$; hard - limiting

Adaline : $\Delta \mathbf{w} = \eta(d - y)\mathbf{x}$; linear function

Delta : $\Delta \mathbf{w} = \eta(d - y)\Phi(I)' \mathbf{x}$; smooth function

*A Degree is not a Degree in Action,
If not for True Learning and Function.*

Thank You